ZHN1 — ZHN1 TASK 1: APPLICATION AND NETWORK LOG EVALUATION

SOFTWARE SECURITY AND TESTING — D385

PRFA — ZHN1

COMPETENCIES

Competency 4102.1.1 : Evaluates Application and Network Logs

The learner evaluates application and network logs for performance, availability, and security vulnerabilities.

Competency 4102.1.2 : Develops Mitigation Solutions

The learner develops mitigation solutions for security vulnerabilities.

Competency 4102.1.3 : Configures Security Authentication

The learner configures security authentication for representational state transfer (REST) and application programming interfaces (APIs).

INTRODUCTION

In this assessment, you will analyze a partially developed Python application to demonstrate your understanding of fundamental software security concepts, including analysis of defensive coding practices, logging information, debugging techniques, and static code analysis. You will modify and test the application by generating detailed log files, identifying and correcting security vulnerabilities, and creating unit tests to validate functionality. You will also analyze log data to identify abnormal or security-relevant patterns and evaluate the overall security of the application. Your findings will be documented in a written report. This assessment reflects real-world responsibilities related to application security monitoring and secure software maintenance.

The final submission must include the following components:

1. Task 1 written report: one Microsoft Word document (.docx) containing the Task 1 written report
2. a ZIP file containing the modified Python application file and a log file

SCENARIO

You are provided with a partially operational Flask-based web application for the WGU construction equipment rental system. While the application successfully loads and processes user requests, it lacks sufficient stability and essential security controls.

Help

An analysis of the existing system logs stored in network_security_log.txt reveals multiple issues. These issues include application crashes caused by runtime errors, security vulnerabilities such as unrestricted user input and requests originating from suspicious IP addresses, and the absence of real-time logging. Together, these deficiencies limit effective system monitoring and increase the risk of future security exploitation.

REQUIREMENTS

Your submission must represent your original work and understanding of the course material. Most performance assessment submissions are automatically scanned through the WGU similarity checker. Students are strongly encouraged to wait for the similarity report to generate after uploading their work and then review it to ensure Academic Authenticity guidelines are met before submitting the file for evaluation. See Understanding Similarity Reports for more information.

Grammarly Note:

Professional Communication will be automatically assessed through Grammarly for Education in most performance assessments before a student submits work for evaluation. Students are strongly encouraged to review the Grammarly for Education feedback prior to submitting work for evaluation, as the overall submission will not pass without this aspect passing. See Use Grammarly for Education Effectively for more information.

Microsoft Files Note:

Write your paper in Microsoft Word (.doc or .docx) unless another Microsoft product, or pdf, is specified in the task directions. Tasks may not be submitted as cloud links, such as links to Google Docs, Google Slides, OneDrive, etc. All supporting documentation, such as screenshots and proof of experience, should be collected in a pdf file and submitted separately from the main file. For more information, please see Computer System and Technology Requirements.

Assessing the Python Application

Download the partially written Python application titled "d385_software_security_and_testing-task-1-student-starter.zip" and unzip the files. Open the application in Visual Studio Code.

A. Prepare a written analysis of the Python application by doing the following:
   1. Describe static code analysis, including its purpose and role in secure software development.
   2. Explain **five** types of issues static code analysis is designed to detect.
   3. Summarize **five** distinct issues or warnings identified during your manual code review of the application, including a brief explanation of why *each* issue is relevant to application security or reliability.

B. Implement testing and log-based analysis by doing the following:
   1. Direct log entries to a file named "Troubleshooting_studentID.log". The log entry should include time, event, action, and the log level. (Eg, INFO, WARNING, ERROR, etc.)
   2. Use a try/except block to handle **two** expected runtime errors, including logging **one** warning and **one** error message. (e.g., InvalidInput, missing files, etc.)

3. Add **two** assertion statements to validate assumptions, and use an info log message to demonstrate that the assertion was successfully captured in the log.
4. Provide a screenshot of the application running.
5. Provide **four** screenshots in your written report that show that **four** unique tests have passed, such as input validation, correct function outputs, or expected exception handling.

C. Write a report regarding the provided starter code application by doing the following:

1. Analyze the log file generated by your updated application by completing the following:
   a. Explain how log entries revealed system issues or unexpected behavior in this application or similar projects.
   b. Describe **two** specific examples of abnormal patterns or potential vulnerabilities identified through log analysis and the findings for *each* example.
2. Provide a debugging overview that addresses the following:
   a. Describe how debugging techniques were used in your Python script, such as breakpoints, logging, or step-by-step execution.
   b. Discuss a specific defect you identified and the corrective fix you implemented for *each* of the **four** unique screenshots documented in part B5.
3. Explain your use of defensive coding in the updated code, including how input validation, exception handling, and assertion statements were used to prevent or mitigate errors and misuse.
4. Summarize what static code analysis can detect within a codebase, such as security vulnerabilities, code issues, or style violations.

D. Acknowledge sources, using in-text citations and references, for content that is quoted, paraphrased, or summarized.

E. Demonstrate professional communication in the content and presentation of your submission.

File Restrictions

File name may contain only letters, numbers, spaces, and these symbols: ! - _ . * ' ( )
File size limit: 200 MB
File types allowed: doc, docx, rtf, xls, xlsx, ppt, pptx, odt, pdf, csv, txt, qt, mov, mpg, avi, mp3, wav, mp4, wma, flv, asf, mpeg, wmv, m4v, svg, tif, tiff, jpeg, jpg, gif, png, zip, rar, tar, 7z

RUBRIC

A1: DESCRIPTION OF STATIC CODE ANALYSIS

NOT EVIDENT	APPROACHING COMPETENCE	COMPETENT
A description of static code analysis is not provided.	The description of static code analysis is inaccurate or does not include its purpose or role in secure software development.	The description of static code analysis is accurate and includes its purpose and role in secure software development.

A2: EXPLANATION OF STATIC CODE ISSUES

NOT EVIDENT	APPROACHING COMPETENCE	COMPETENT

An explanation of issues detected by static code analysis is not provided.	The explanation includes fewer than 5 types of issues, or 1 or more issue types are not accurately explained.	The explanation includes 5 types of issues detected by static code analysis, and *each* type is accurately explained.

A3:SUMMARY OF IDENTIFIED ISSUES

NOT EVIDENT	APPROACHING COMPETENCE	COMPETENT
A summary of identified issues or warnings is not provided.	The summary includes fewer than 5 distinct issues or warnings, or 1 or more explanations are not relevant to application security or reliability.	The summary includes 5 distinct issues or warnings identified during manual code review, and *each* explanation is relevant to application security or reliability.

B1:LOG FILE CREATION

NOT EVIDENT	APPROACHING COMPETENCE	COMPETENT
A log file is not provided.	The log file is not named "Troubleshooting_studentID.log" or does not capture log entries.	The log file is named "Troubleshooting_studentID.log" and captures log entries.

B2:RUNTIME ERROR HANDLING

NOT EVIDENT	APPROACHING COMPETENCE	COMPETENT
The handling of runtime errors is not provided.	The use of try/except blocks does not handle 2 expected runtime errors, or the logging does not include 1 warning and 1 error message.	The use of try/except blocks handles 2 expected runtime errors, and the logging includes 1 warning and 1 error message.

B3:ASSERTION STATEMENTS

NOT EVIDENT	APPROACHING COMPETENCE	COMPETENT
Assertion statements are not provided.	Only 1 assertion statement is included, or 1 or *both* do not validate an assumption, or an info log message is not used to demonstrate that the assertion	2 assertion statements are included, *each* validates an assumption, and an info log message is used to demonstrate that the assertion was successfully captured in the log.

was successfully captured in the log.

B4: APPLICATION SCREENSHOT

NOT EVIDENT	APPROACHING COMPETENCE	COMPETENT
A screenshot of the application is not provided.	The screenshot does not show the application running.	The screenshot shows the application running.

B5: TEST DOCUMENTATION

NOT EVIDENT	APPROACHING COMPETENCE	COMPETENT
A screenshot is not provided.	The report includes screenshots of fewer than 4 unique passed tests, or 1 or more screenshots are not relevant to input validation, function outputs, or exception handling.	The report includes screenshots of 4 unique passed tests, and *each* is relevant to input validation, correct function outputs, or expected exception handling.

C1A: LOG EXPLANATION

NOT EVIDENT	APPROACHING COMPETENCE	COMPETENT
An explanation of how log entries revealed issues is not provided.	The explanation does not connect log entries to system issues or unexpected behavior, or the explanation is not appropriate for this application or similar projects.	The explanation connects log entries to system issues or unexpected behavior, and the explanation is appropriate for this application or similar projects.

C1B: LOG ANALYSIS EXAMPLES

NOT EVIDENT	APPROACHING COMPETENCE	COMPETENT
A description of examples from log analysis is not provided.	The description includes fewer than 2 specific examples of abnormal patterns or potential vulnerabilities, or the findings are not explained or are not appropriate.	The description includes 2 specific examples of abnormal patterns or potential vulnerabilities identified through log analysis, and the findings for *each* example are explained and appropriate.

C2A: DEBUGGING TECHNIQUES

NOT EVIDENT	APPROACHING COMPETENCE	COMPETENT
A description of debugging techniques is not provided.	The description of debugging techniques does not describe how techniques such as breakpoints, logging, or step-by-step execution were used, or the techniques are not appropriate for the script.	The description of debugging techniques describes how techniques such as breakpoints, logging, or step-by-step execution were used, and the techniques are appropriate for the script.

C2B: DEBUGGING FIXES

NOT EVIDENT	APPROACHING COMPETENCE	COMPETENT
A discussion of the defects and fixes from the screenshots in part B5 is not provided.	The discussion does not include a specific defect and corrective fix for *each* of the 4 screenshots documented in part B5, or 1 or more fixes are not appropriate.	The discussion includes a specific defect and a corrective fix for *each* of the 4 screenshots documented in part B5, and *each* fix is appropriate.

C3: DEFENSIVE CODING EXPLANATION

NOT EVIDENT	APPROACHING COMPETENCE	COMPETENT
An explanation of defensive coding is not provided.	The explanation does not address 1 or more listed elements (input validation, exception handling, or assertion statements), or 1 or more elements are not connected to preventing or mitigating errors or misuse.	The explanation addresses input validation, exception handling, and assertion statements and connects *each* to preventing or mitigating errors or misuse.

C4: STATIC CODE ANALYSIS SUMMARY

NOT EVIDENT	APPROACHING COMPETENCE	COMPETENT
A summary of what static code analysis can detect is not provided.	The summary does not describe how static code analysis can detect security vulnerabilities, code issues, or style violations, or 1 or more elements are missing or inaccurate.	The summary accurately describes how static code analysis can detect security vulnerabilities, code issues, and style violations.

D: SOURCES

NOT EVIDENT	APPROACHING COMPETENCE	COMPETENT
The submission does not include both in-text citations and a reference list for sources that are quoted, paraphrased, or summarized.	The submission includes in-text citations for sources that are quoted, paraphrased, or summarized and a reference list; however, the citations or reference list is incomplete or inaccurate.	The submission includes in-text citations for sources that are properly quoted, paraphrased, or summarized, and a reference list that accurately identifies the author, date, title, and source location as available.

E:PROFESSIONAL COMMUNICATION

NOT EVIDENT	APPROACHING COMPETENCE	COMPETENT
This submission includes pervasive errors in professional communication related to grammar, sentence fluency, contextual spelling, or punctuation, negatively impacting the professional quality and clarity of the writing. Specific errors have been identified by Grammarly for Education under the Correctness category.	This submission includes substantial errors in professional communication related to grammar, sentence fluency, contextual spelling, and/or punctuation. Specific errors have been identified by Grammarly for Education under the Correctness category.	This submission includes satisfactory use of grammar, sentence fluency, contextual spelling, and punctuation, which promote accurate interpretation and understanding.

WEB LINKS

Visual Studio Code

SUPPORTING DOCUMENTS

d385_software_security_and_testing-task-1-student-starter.zip